

Data types

- Data in the form text
 - strings
 - Boolean
- Data in the form of numbers
 - integer
 - binary
 - octa
 - hexa
 - float
 - Complex

Integers

```
In [1]: num=100  
        type(num)
```

```
Out[1]: int
```

binary

- it is also a number format
- bi means 2
- it has combination of only two digits
- 0 1 2 3 4 5 6 7 8 9
- binary representation always in the form of 0 and 1
- Example: 0b111, 0b101, 0B101

```
In [3]: 0b101
```

```
Out[3]: 5
```

```
In [ ]: 0b1110
```

```
In [ ]: whats difference between 0b101 and 101
```

```
In [4]: 101
```

```
Out[4]: 101
```

```
In [ ]: 0b101
```

```
In [ ]: STLD
        DLD
```

```
In [ ]: 4 2 1
        0 0 0===== 0
        0 0 1===== 1
        0 1 0===== 2
        0 1 1===== 3
        1 0 0===== 4
        1 0 1===== 5
        1 1 0===== 6
        1 1 1===== 7
```

```
In [7]: 0b010
```

```
Out[7]: 2
```

octa

it is also a number format

octa means 8

it has combination of only 8 digits

0 1 2 3 4 5 6 7

binary representation always in the form of 0 to 7

Example: 0o<0 to 7> 0O<0to7>

: 0o456 valid

0o1234 valid

0o786 notvalid

0o123

```
In [8]: 0o123
```

```
Out[8]: 83
```

```
In [9]: name="danish"
        type(name)
```

```
Out[9]: str
```

```
In [10]: num=100
         type(num)
```

```
Out[10]: int
```

In [1]: `0o786`

```
Cell In[1], line 1
    0o786
      ^
SyntaxError: invalid digit '8' in octal literal
```

In []: hexa

it **is** also a number **format**

hexa means **16**

it has combination of only **16** digits

0 1 2 3 4 5 6 7 8 9 A(10) B(11) C(12) D(13) E(14) F(15)

Example: **0x<0 to 9 and A to F>** **0X<0to9 and A to F>**

```
0xabc    valid
0x123    valid
0x11a09h not valid
```

In [2]: `0x123`

Out[2]: 291

- you can provide any format of input bin/oct/hex
- the output will be decimal

In [3]: `type(0x123)`

Out[3]: int

In [4]: `type(0b111)`

Out[4]: int

In [5]: `type(0o123)`

Out[5]: int

In [6]: `type(100)`

Out[6]: int

float

```
In [ ]: abc ===== which type

a) bin b) oct c)hex =====

why it is hexa
A to F range

hex definition range 0 to F

hex : number 16

How many digits we required : 16 digits
is 16 digits are available : 0 to 9 A(10) B(11) C(12) D(13) E(14) F(15)

bin : 2

oct : 8
```

```
In [7]: 0xabc
```

```
Out[7]: 2748
```

```
In [8]: type(10.5)
```

```
Out[8]: float
```

```
In [9]: salary=12345.6
type(salary)
```

```
Out[9]: float
```

- some float type measurements
 - height = 5.2
 - weight = 60.2
 - distance = 25.6cm
- some integer types:
 - how many cars you have
 - how many house you have

strings

```
In [10]: name='python'
print(name)
```

```
python
```

```
In [11]: type(name)
```

```
Out[11]: str
```

```
In [ ]: int float str
```

```
In [14]: name='python'  # single quotes
         name1="hyd"    # double quotes
         print(name,name1)
```

python hyd

```
In [ ]: I like 'python'
```

```
In [19]: string1='I like "python"'
         print(string1)
```

I like "python"

- both single quotes and double quotes will work
- if you want to highlight any specific word in a string
 - provide entire string double quotes
 - provide single quotes to a word which you want highlight
 - 'I like "python"' or "I like 'python'"
 - vice versa

doc string

```
In [ ]: """
         hello how are you
         im good
         how about you
         """
```

```
In [20]: """
         i have taken a=10
         i have taken b=20
         then i added a with b
         that i saved in c
         """

         a=10
         b=20
         c=a+b
```

- python code you can write using single and double quotes
- but triple quotes you are writing means
- you are trying convey some information to user

boolean

```
In [ ]: val=true
```

```
In [ ]: true
        True
        "True"
```

```
In [21]: True
```

```
Out[21]: True
```

```
In [22]: "true"
```

```
Out[22]: 'true'
```

```
In [23]: true
```

```
-----
-
NameError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 true

NameError: name 'true' is not defined
```

```
In [24]: val=True
        type(val)
```

```
Out[24]: bool
```

```
In [ ]: int float str bool bin oct hex
```

```
In [25]: val2=False
        type(False)
```

```
Out[25]: bool
```

```
In [ ]: True ==== > 1
        False ==== > 0
```

```
In [26]: false=100 # 100 value saving in a variable false
        val=False
        name='False'
        type(false), type(val), type(name)
```

```
Out[26]: (int, bool, str)
```

```
In [27]: type(100), type(False), type("False")
```

```
Out[27]: (int, bool, str)
```

```
In [28]: type(false=100)
```

```
-----  
-  
TypeError                                Traceback (most recent call las  
t)  
Cell In[28], line 1  
----> 1 type(false=100)  
  
TypeError: type() takes 1 or 3 arguments
```

```
In [30]: false=100  
type(false)      # works  
  
type(100)        # works  
  
type(false=100)  # fail  
  
type(false,100)  # fail
```

```
Out[30]: int
```

```
In [32]: type(false)
```

```
Out[32]: int
```

```
In [36]: false=100  
print(false)      # works  
  
print(100)        # works  
  
print(false=100)  # fail  
  
print(false,100)  # works
```

```
100  
100
```

```
-----  
-  
TypeError                                Traceback (most recent call las  
t)  
Cell In[36], line 7  
      2 print(false)      # works  
      5 print(100)        # works  
----> 7 print(false=100)  # fail  
      9 print(false,100)  
  
TypeError: 'false' is an invalid keyword argument for print()
```

```
In [35]: name1='hello'
name2='python'
print('hello',name2)
```

hello python

```
In [ ]: type(a=6)
print(b=7)
```

Complex

- complex number format a+ib or a+jb
- where a= real number
- b= imaginary number

```
In [37]: num=10+20j
type(num)
```

Out[37]: complex

```
In [ ]: # apply dir on num
# see the methods
# real
# img
```



```
In [38]: dir(num)
```

```
Out[38]: ['__abs__',
          '__add__',
          '__bool__',
          '__class__',
          '__complex__',
          '__delattr__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattr__',
          '__getnewargs__',
          '__getstate__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__le__',
          '__lt__',
          '__mul__',
          '__ne__',
          '__neg__',
          '__new__',
          '__pos__',
          '__pow__',
          '__radd__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rmul__',
          '__rpow__',
          '__rsub__',
          '__rtruediv__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__sub__',
          '__subclasshook__',
          '__truediv__',
          'conjugate',
          'imag',
          'real']
```

```
In [47]: num=10+20j
real=num.real
img=num.imag
conj=num.conjugate()

print(f"real value of {num} is {real}")
print(f"imaginary value of {num} is {img}")
print(f"conjugate value of {num} is {conj}")
```

```
real value of (10+20j) is 10.0
imaginary value of (10+20j) is 20.0
conjugate value of (10+20j) is (10-20j)
```

```
In [52]: num.conjugate() # syntax

#function
#bound method you need provide the brackets
```

Out[52]: (10-20j)

```
In [57]: import random
random.randint()

# take your cursor inside the bracket
# apply shift+tab
```

```
-----
-
TypeError                                Traceback (most recent call las
t)
Cell In[57], line 2
      1 import random
----> 2 random.randint()

TypeError: Random.randint() missing 2 required positional arguments: 'a' a
nd 'b'
```

```
In [58]: import math
```

```
In [62]: math.sin(90)
```

Out[62]: 0.8939966636005579

```
In [63]: math.sin()
```

Out[63]: <function math.sin(x, /)>

```
In [65]: num.conjugate()
```

Out[65]: (10-20j)

```
In [ ]:
```